

Documentación Técnica y Funcional Profunda: KeySearch V6.0

Análisis Explicativo y Lógica de Negocio de cada Componente

Este documento está diseñado para entender a fondo la arquitectura, las lógicas internas de programación y los algoritmos que componen la herramienta **KeySearch Diarrea de Perro V6.0**. Está escrito para que sea comprensible, revelando el "por qué" y el "cómo" detrás de cada archivo.

1. Arquitectura General del Proyecto

La herramienta está estructurada en módulos independientes para garantizar la escalabilidad y mantenibilidad. El flujo de datos sigue un modelo de "Pipeline" (tubería):

1. **Entrada de Datos:** El usuario ingresa variables (keywords, país, perfil).
 2. **Scraping y Extracción:** Los módulos extraen datos brutos de la web mediante HTTP puro.
 3. **Limpieza y Enriquecimiento:** Se filtran datos inservibles (IA) y se cruzan con APIs oficiales (Google Ads y Trends).
 4. **Exportación:** Se formatea y consolida todo en una hoja de Excel corporativa y un archivo JSON.
-

2. Análisis Archivo por Archivo

main.py (Orquestador Principal)

Este es el punto de entrada. No hace scraping ni exporta por sí mismo; actúa como el "Controlador" del patrón MVC (Modelo-Vista-Controlador).

- **Flujo de Interfaz y Rich:** Usa la librería `rich` para generar una interfaz de consola amigable, con tablas, barras de carga y colores.
- **Gestión de `sys.argv` y CLI:** A través de la librería `argparse`, soporta ejecución en modo silencioso (por ejemplo, pasándole un archivo txt con `--file`), permitiendo automatizar lotes enteros sin intervención manual.
- **`buscar_keyword()`:** Es el corazón del archivo. Aquí se define el orden cronológico estricto:

1. Llama a `get_autocomplete_suggestions`.
2. Llama a `get_question_suggestions`.
3. Llama a `scrape_google` (donde ocurre la conexión directa a la SERP).
4. Ejecuta `filtrar_con_ia` si la llave de Groq está configurada.
5. Llama a `estimar_volumenes` para obtener datos de Trends.
6. Llama a `enrich_with_google_ads_metrics`.

config.py (Centro de Configuración y Constantes)

Almacena todas las variables de entorno, tiempos de espera (delays) y diccionarios globales.

- **Anti-Detección y USER_AGENT_PROFILES:** Google rastrea si las peticiones automatizadas tienen encabezados HTTP inconsistentes. Este archivo no solo guarda User-Agents falsos (cadenas que dicen "soy Chrome en Windows"), sino que los empareja con la cabecera `sec-ch-ua` (Client Hints) correspondiente. Si estos no coinciden, Google bloquea la petición inmediatamente.
 - **Perfiles de Scraping (Normal vs Extreme):** Dependiendo del perfil seleccionado, las constantes como `AUTOCOMPLETE_DEEP_EXPANSION_LIMIT` o `SERP_PAGES` se multiplican. El perfil extremo no cambia la forma de extraer, sino la **profundidad de recursividad** de los algoritmos.
 - **Fusión de BASE_DIR y PyInstaller:** Usa la función `_runtime_base_dir()` para asegurar que cuando el programa se convierta en un `.exe`, logre encontrar plantillas de Excel y archivos locales (usando el directorio temporal `_MEIPASS`).
-

Carpeta scraper (Motores de Extracción)

1. `google_serp.py` (Módulo de Extracción de SERP HTML)

Su trabajo es enviar una petición HTTP al buscador de Google y descargar el HTML resultante para extraer el bloque PAA (People Also Ask) y las Búsquedas Relacionadas.

- **Evadiendo el Error 505 y Bloqueos:**
 - La función `_crear_sesion()` fuerza la conexión HTTP/1.1 (`Connection: keep-alive`). Google a menudo rechaza peticiones HTTP/2 enviadas desde la librería `requests` porque detecta que no es un navegador real.
 - **Cooldowns 429:** Si Google devuelve un error HTTP 429 (Demasiadas peticiones), la lógica implementa un *Breaker* exponencial, pausando la extracción durante 2 a 4 minutos para "enfriar" la IP.
- **Parsing Resiliente de HTML:** Google cambia el código HTML de su página casi a diario. La función `_extraer_preguntas_paa_html` usa **6 estrategias distintas** con `BeautifulSoup` para buscar las preguntas PAA. Busca atributos como `data-sgrd`, `related-question-pair` y hasta explora dentro del código JavaScript (JSON-LD) incrustado en la página para robar las preguntas antes de que se rendericen.
- **Técnica de Fallback a Autocomplete:** Si Google bloquea la lectura del HTML o no muestra la caja de PAA, el algoritmo no se rinde. Llama a `_extraer_preguntas_paa_autocomplete()`, que simula preguntas escribiendo la keyword junto con modificadores ("cómo [keyword]", "qué es [keyword]") en la barra de autocompletado para deducir qué preguntas mostraría Google.

2. `autocomplete.py` (Módulo de Expansión por Autocompletado)

Ataca el endpoint no documentado (semi-público) de Google Suggest (`suggestqueries.google.com`).

- **El Método del Alfabeto:** La función `get_autocomplete_suggestions()` concatena la keyword principal con todas las letras del alfabeto (Ej: "Keyword a", "Keyword b") y procesa la respuesta en formato JSON.
- **Recursividad Profunda:** En el perfil extremo, no se detiene en el primer nivel. Toma las 10 mejores sugerencias encontradas y **vuelve a pasarlas** por el autocompletado

como si fueran semillas. Esto genera un árbol de sugerencias que descubre nichos muy específicos de "Long Tail" (Larga Cola).

3. `volume_estimator.py` (Motor de Scoring y Google Trends)

Eliminó la antigua práctica de "inventar" volúmenes de búsqueda para adoptar un modelo de datos reales y priorización matemática.

- **Score Matemático por Posición:** La función `_score_por_posicion` usa la fórmula $(ratio^{*0.7}) * 100 * peso_fuente$.
 - Si una pregunta está en el puesto #1, recibe casi 100 puntos.
 - Si está de última, el decaimiento no es lineal (gracias a la potencia 0.7), lo que asegura que las palabras en medio de la lista aún conserven un puntaje decente.
 - Se multiplica por un `peso_fuente` (Autocompletado vale 1.0, pero Relacionadas vale 0.65).
- **Integración con Google Trends (`pytrends`):** Toma las mejores palabras clave y las consulta en lotes de 5 en Google Trends. Extrae el promedio de interés relativo (de 0 a 100) en los últimos 12 meses.
- **Fusión de Scores:** Combina el puntaje de posición interna (40% de peso) con el interés real de Trends (60% de peso) para definir si la prioridad final es "Alta", "Media" o "Baja".

4. `google_ads_metrics.py` (Conexión Oficial a Google Ads API)

Se conecta a la infraestructura oficial de Google Ads usando autenticación OAuth2 y el Customer ID.

- **Resolución de Targets Geográficos:** Convierte el código de país (ej. "CO") en un Resource Name exacto de Google Ads (ej. `geoTargetConstants/2170`) mediante la API.
- **Cruce con Variantes Cercanas:** La API de Ads suele agrupar palabras (ej. "zapatos de hombre" y "zapato de hombre" devuelven una sola métrica). El script asocia astutamente el resultado devuelto con todas las variaciones locales (`close_variants`) detectadas. Extrae promedios mensuales y métricas de puja (CPC alto y bajo).

5. `categorizer.py` (Motor de Taxonomía y NLP Básico)

Acomoda las palabras en categorías editoriales predefinidas de forma offline, sin usar APIs.

- **Taxonomía en Duro:** Posee una lista en memoria (`_TAXONOMY`) de categorías (Salud, Negocios, etc.) y subcategorías, cada una con un banco de palabras clave.
- **Sistema de Puntaje Normalizado:** Aplica normalización Unicode (eliminando tildes). Si la keyword coincide exactamente con una palabra clave de la taxonomía, suma 3 puntos. Si la contiene parcialmente, suma 1 punto. Selecciona la categoría con mayor puntaje (mayor a 0.5) para etiquetar el reporte.

6. `ai_filter.py` (Motor de IA Generativa con Groq)

Agrega una capa de inteligencia semántica para limpiar el "ruido" que Google siempre devuelve.

- **Limpieza de Datos:** Envía el listado gigante a la API de Groq (usando Llama 3). A través del Prompt, le indica que elimine cualquier palabra clave con intención geográfica de un país ajeno al analizado, y palabras incoherentes, exigiendo la salida en **formato JSON estricto**.
- **Generación Editorial:** Analizando el Top 5 de las búsquedas extraídas, pide a la IA que deduzca qué tema agrupa todo el reporte y genere 5 propuestas de Títulos SEO y enfoques de artículo.

7. `http_cache.py` y `utils.py` (Utilidades Criptográficas y de Limpieza)

- **Hashing de Caché:** `make_key()` toma cualquier URL y la codifica mediante el algoritmo SHA256. Esto crea un nombre de archivo único para guardar temporalmente la respuesta de esa petición. Si en 24 horas el script pide la misma URL exacta, la lee del disco, ahorrando una petición de red.
- **Normalización Estricta:** `dedupe_key()` en `utils.py` limpia toda la basura: remueve caracteres especiales, acentos y espacios dobles, asegurando que "perro" y "pérro" se contabilicen como la misma entidad para eliminar duplicados de la lista final.

Carpeta `exporters` (Módulos de Formateo Final)

1. `excel_export.py` (Renderizado Avanzado de Reportes)

Usa `openpyxl` para inyectar datos en una plantilla prediseñada o crear un archivo desde cero.

- **Mapeo de Coordenadas:** Para la hoja de "Resumen", asigna los datos de volumen, categorías, y los bloques editoriales (títulos de IA) a filas y columnas específicas (ej. Fila 14 Columna 2).
- **Inyección Condicional de Estilos:** Al crear las hojas de datos tabulares, no aplica un color base estándar. Lee el score de la palabra clave calculada, y dinámicamente inyecta colores rojo oscuro (`FILL_SCORE_VERY_HIGH`) a los temas urgentes y grises a los temas secundarios, permitiendo un escaneo visual instantáneo para el analista.

2. `json_export.py` (Serialización de Estructuras)

Crea una huella del análisis en formato JSON estandarizado, separando claramente los metadatos de metodología de las listas serializadas. Útil si esta herramienta se llegara a usar como un servicio backend (API) para conectarlo a una base de datos externa.

Conclusión

KeySearch V6.0 ha evolucionado de un simple scraper a un complejo ensamblaje de inteligencia de datos. Cruza análisis de expresiones regulares y algoritmos heurísticos locales (Categorizador, Hashing), extracción de dominios externos inestables (SERP y Autocompletado), validación analítica en frío (APIs de Google Ads/Trends) y modelado de datos semánticos con LLMs (Groq), finalizando en presentaciones ejecutivas. Todo ello, esquivando de forma matemática los sistemas anti-bots de Google.